

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

# 拓扑排序

## Topological Sort: Kahn and DFS Approaches

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 30 日

# 目录

1. 拓扑排序概述
2. Kahn 算法
3. DFS 版拓扑排序
4. 判环
5. 总结



# 目录

1 拓扑排序概述

2 Kahn 算法

3 DFS 版拓扑排序

4 判环

5 总结

# 什么是拓扑排序



**拓扑排序**：把一个有向图的顶点排成一个线性序列，使每条边 $(u, v)$ 都满足 $u$ 排在 $v$ 之前。

**例子**：边 $1 \to 2$，$2 \to 3$

- **合法序列**：$1,\ 2,\ 3$
- **不合法**：$2,\ 1,\ 3$（1 必须在 2 前）
- **不合法**：$1,\ 3,\ 2$（2 必须在 3 前）

拓扑序：所有边都从前往后指

# 有向无环图（DAG）



**前提**：拓扑排序只对有向无环图（DAG）存在解。

**入度与出度**

- 入度：有几条边指向该点
- 出度：有几条边从该点出发

**为什么有环就无解**：若有 $a \to b \to c \to a$，则三条先后要求互相矛盾。

无环必有拓扑序，有环一定无解

# 拓扑排序的典型应用

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

**三类常见场景**

- 课程先修关系：先修课排在后继课之前
- 任务依赖调度：被依赖的任务先执行
- 编译构建顺序：被 include 的头文件先编译

**两种主流算法**

- Kahn 算法：按入度为 0 逐个剥离，BFS 风格
- DFS 算法：后序遍历的逆序就是拓扑序

同一张 DAG，两种视角都能排出拓扑序

# 目录

1 拓扑排序概述

2 Kahn 算法

3 DFS 版拓扑排序

4 判环

5 总结

# 例题：课程安排



**题目**：有 $n$ 门课程、$m$ 条先修关系。每条关系 $(u, v)$ 表示“课程 $u$ 必须在课程 $v$ 之前修”。求一个合法的修课顺序；若无解，输出“存在环”。

**示例**

- $n = 4$，关系 $(1, 2)$、$(1, 3)$、$(2, 4)$、$(3, 4)$
- 一种合法顺序：$1,\ 2,\ 3,\ 4$
- 另一种合法顺序：$1,\ 3,\ 2,\ 4$

# 课程安排：Kahn 算法解法



**核心思路**：每次挑一个入度为 0 的点输出，然后把它的出边剪掉，后继入度减 1。

**关键步骤**

- 第一步：计算所有点的入度，把入度为 0 的点全部入队
- 第二步：每次弹出队首 $u$，把 $u$ 加入答案序列
- 第三步：对 $u$ 的每条出边 $(u, v)$，把 $v$ 的入度减 1
- 第四步：若 $v$ 的入度减到 0，就把 $v$ 入队
- 队列空时，答案长度等于 $n$ 则无环，否则有环

**为什么正确**：入度为 0 表示没有前置约束，可以放心输出；每轮至少剥掉一层，有限步内一定结束。

# Kahn 算法：模拟执行



**图**：边 $1 \to 2$、$1 \to 3$、$2 \to 4$、$3 \to 4$，初始入度为 $0, 1, 1, 2$。

| 步骤 | 弹出 | 输出序列 | 入度变化 |
|---|---|---|---|
| 初始 | – | 空 | 队列 $\{1\}$ |
| 1 | 1 | 1 | $in[2] = 0$，$in[3] = 0$，队 $\{2, 3\}$ |
| 2 | 2 | 1, 2 | $in[4] = 1$，队 $\{3\}$ |
| 3 | 3 | 1, 2, 3 | $in[4] = 0$，队 $\{4\}$ |
| 4 | 4 | 1, 2, 3, 4 | 队空，结束 |

输出长度 $= 4 = n$，说明无环，拓扑序为 $1,\ 2,\ 3,\ 4$。

# Kahn 算法代码



```cpp
vector<int> g[MAXN];
int in[MAXN];
vector<int> topo;
bool kahn(int n) {
    queue<int> q;
    for (int i = 1; i <= n; i++)
        if (in[i] == 0) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        topo.push_back(u);
        for (int v : g[u]) {
            if (--in[v] == 0) q.push(v);
        }
    }
    return (int)topo.size() == n;
}
```

**复杂度：每个点出队一次，每条边处理一次，总时间 $O(n+m)$。**

# 目录

1 拓扑排序概述

2 Kahn 算法

3 DFS 版拓扑排序

4 判环

5 总结



# DFS 版：思路与步骤



**核心思路**：对每个点 DFS，某点“访问完所有后继”后压入结果序列，最后整体反转。

**关键步骤**

- 第一步：用三色标记：白色未访问、灰色正在访问、黑色已完成
- 第二步：从任意白点出发 DFS，进入时标灰
- 第三步：遍历所有后继后，把当前点压入栈，标黑
- 第四步：DFS 遇到灰色点说明有环，直接判无解
- 第五步：所有点处理完后，反转栈得到拓扑序

**为什么正确**：从 $u$ 出发 DFS，$u$ 的所有后继都比 $u$ 先完成，反转后 $u$ 就跑到后继前面，正好满足边 $(u, v)$。

# DFS 版代码



```
vector<int> g[MAXN];
int color[MAXN];        // 0 白, 1 灰, 2 黑
vector<int> topo;
bool ok = true;
void dfs(int u) {
    color[u] = 1;
    for (int v : g[u]) {
        if (color[v] == 1) { ok = false; return; }
        if (color[v] == 0) {
            dfs(v);
            if (!ok) return;
        }
    }
    color[u] = 2;
    topo.push_back(u);
}
void topo_sort(int n) {
    for (int i = 1; i <= n; i++)
```

# DFS 版代码

```
19          if (color[i] == 0) dfs(i);
20      reverse(topo.begin(), topo.end());
21  }
```

**复杂度 $O(n+m)$；三色标记顺带判环。**

# 目录

1 拓扑排序概述

2 Kahn 算法

3 DFS 版拓扑排序

4 判环

5 总结



# 判环：两种视角



**Kahn 视角**

- 有环时，环上的点入度永远不为 0，无法入队
- 最终输出点数小于 $n$，即判为有环

**DFS 视角**

- DFS 时若遇到正在访问（灰色）的点，说明回到了自己
- 这条回边就是环，直接判为有环

Kahn 看输出长度，DFS 看灰色回边

# 例题：判断有向图是否有环



**题目**：给定 $n$ 个点、$m$ 条有向边。判断图中是否存在环。若无环，输出任意一个拓扑序；若有环，输出“存在环”。

**示例**

- 边 $1 \to 2$、$2 \to 3$、$3 \to 1$：有环，输出“存在环”
- 边 $1 \to 2$、$1 \to 3$、$2 \to 4$：无环，可输出 $1,\ 3,\ 2,\ 4$
- 无边（$m = 0$）：无环，输出 $1,\ 2,\ \ldots,\ n$

# 判断有环：解法



**核心思路**：用 Kahn 算法跑一遍，看输出长度是否等于 $n$。

**关键步骤**

- 第一步：计算所有点入度，入度为 0 的点全部入队
- 第二步：不断弹出队首并输出，后继入度减 1
- 第三步：若减到 0 就入队；队列空时检查输出数量
- 第四步：输出数量等于 $n$ 则无环，否则有环

**为什么能判环**：环上每个点入度至少为 1，永远轮不到它们入队，所以有环时答案长度严格小于 $n$。

# 目录

1 拓扑排序概述

2 Kahn 算法

3 DFS 版拓扑排序

4 判环

5 总结



# 总结（一）：Kahn vs DFS

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

| 对比项 | Kahn 算法 | DFS 算法 |
|---|---|---|
| 数据结构 | 队列 + 入度数组 | 递归 + 三色标记 |
| 思路 | 从入度 0 剥洋葱 | 后序压栈后反转 |
| 判环 | 输出长度小于 $n$ | 遇到灰色回边 |
| 复杂度 | $O(n+m)$ | $O(n+m)$ |
| 适用场景 | 迭代实现、避栈溢出 | 顺带判环、代码更短 |

同一张 DAG，两种写法，答案都对

# 总结（二）：常见陷阱

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

**实现细节**

- 忘记把初始入度 0 的所有点都入队（漏解）
- 每次 $in[v]$ 减 1 后，忘了判是否为 0 就入队
- 判环时只看队列空不空，忘了比较输出长度与 $n$

**DFS 版特有**

- 忘记反转，得到的是逆拓扑序
- 未用灰色标记判环，遇到环时会死循环

入队别漏、减完再判、最后对长度

# 附录：拓扑序不唯一



**为什么同一个 DAG 可能有很多种拓扑序**

- 若多个点同时入度为 0，它们的相对顺序可以任意
- Kahn 用队列时，弹出顺序取决于入队顺序
- 用优先队列（最小堆），就能得到字典序最小的拓扑序

**例子**：边 $1 \to 3$、$2 \to 3$

- 合法拓扑序有：$1, 2, 3$ 和 $2, 1, 3$
- 用最小堆：得到 $1, 2, 3$

要字典序最小，把队列换成优先队列

# 附录：复杂度对比



| 项目 | Kahn 算法 | DFS 算法 |
| --- | --- | --- |
| 时间 | $O(n+m)$ | $O(n+m)$ |
| 空间 | $O(n+m)$ | $O(n+m)$ |
| 递归深度 | 无递归 | 最坏 $O(n)$ |
| 字典序优化 | 换优先队列，$O(n \log n)$ | 不便优化 |

两者时间、空间同阶，选顺手的写法就行